

Intrusive and Non-Intrusive Model Reduction

Homework 1 – Thermal block: full model, reduction, and UQ

Due: Friday, 16 October 2026, 08:00

Reproducibility and ownership of the computation

The routine implementation in Exercise 1.1 is intentionally compact. The assessed part of the homework is the numerical evidence and the mathematical interpretation in Exercises 1.2–1.4. You are responsible for being able to explain every figure and conclusion in your submission.

Computing

Use JAX throughout. The full-order stiffness matrix must *not* be assembled as a dense or sparse matrix. In particular, do not use SciPy sparse matrices. Apply the stiffness operator matrix-free and solve with `jax.scipy.sparse.linalg.cg`, passing the operator as a callable. Use double precision via `jax.config.update("jax_enable_x64", True)` and use `jax.jit` for the repeated model evaluations. JAX's CG accepts a callable matrix-vector product and supports differentiation by implicit differentiation. See the current JAX documentation.

What to submit

Submit

- one single PDF report containing exactly the requested derivations, plots, tables, and discussions.
- the complete source code.
- a README with the commands needed to reproduce each figure.
- the raw numerical data used for the figures.

Unless otherwise stated, a discussion of 150–250 words is sufficient.

Exercise 1.1: Full-order model: a reusable matrix-free JAX implementation

Let $\Omega = (0, 1)^2$ and consider

$$\begin{aligned}
 -\nabla \cdot (\kappa(x; \mu) \nabla u(x; \mu)) &= 0 && \text{in } \Omega, \\
 u(x; \mu) &= 0 && \text{on } \Gamma_D = \{x_2 = 0\}, \\
 \kappa \nabla u \cdot n &= 1 && \text{on } \Gamma_N = \{x_2 = 1\}, \\
 \kappa \nabla u \cdot n &= 0 && \text{on } \Gamma_0 = \{x_1 = 0\} \cup \{x_1 = 1\}.
 \end{aligned}$$

Here V denotes the space of admissible test and trial functions that vanish on Γ_D . The weak problem is: find $u(\mu) \in V$ such that

$$a(u(\mu), v; \mu) = f(v) \quad \text{for all } v \in V,$$

where

$$a(w, v; \mu) = \int_{\Omega} \kappa(x; \mu) \nabla w \cdot \nabla v \, dx, \quad f(v) = \int_{\Gamma_N} v \, ds.$$

For

$$p \in \{2, 4, 9, 16, 25\}$$

use the block layouts

$$(B_x, B_y) = (2, 1), (2, 2), (3, 3), (4, 4), (5, 5), \quad p = B_x B_y.$$

The blocks are numbered row-wise from the bottom left. The conductivity is

$$\kappa(x; \mu) = \sum_{q=1}^p \mu_q \chi_{\Omega_q}(x), \quad \mu \in \mathcal{D}_p = [0.1, 10]^p.$$

Unless explicitly stated otherwise, use the default parameter dimension $p = 9$.

At level ℓ , use

$$n = 2^\ell$$

rectangular elements in each coordinate direction and mesh width $h = 1/n$. The finite-element basis consists of tensor products of one-dimensional hat functions,

$$\varphi_{ij}(x_1, x_2) = \ell_i(x_1) m_j(x_2),$$

with the basis functions on Γ_D removed. Thus

$$N = n(n+1) = 2^\ell(2^\ell + 1)$$

unknowns remain.

For $p = 9$ and $p = 25$, block interfaces do not generally coincide with mesh lines because 2^ℓ is not divisible by 3 or 5. To keep the full-model implementation simple, assign each finite element to the thermal block containing its midpoint. The resulting discrete coefficient is therefore constant on each element and converges to the intended thermal-block coefficient as ℓ is refined.

Number the four vertices of one element counter-clockwise starting at the lower-left corner. For a square element the parameter-independent local stiffness and mass matrices are

$$K_{\text{loc}} = \frac{1}{6} \begin{bmatrix} 4 & -1 & -2 & -1 \\ -1 & 4 & -1 & -2 \\ -2 & -1 & 4 & -1 \\ -1 & -2 & -1 & 4 \end{bmatrix}, \quad M_{\text{loc}} = \frac{h^2}{36} \begin{bmatrix} 4 & 2 & 1 & 2 \\ 2 & 4 & 2 & 1 \\ 1 & 2 & 4 & 2 \\ 2 & 1 & 2 & 4 \end{bmatrix}.$$

Use these formulas directly. No element-matrix derivation is required in this homework.

The discrete operator has the affine form

$$A(\mu) = \sum_{q=1}^p \mu_q A_q,$$

but the matrices A_q must not be assembled. Instead, your model should store the element connectivity and the element-to-block map and provide matrix-free applications of $A(\mu)$, A_q , and the mass matrix M . The coefficient-vector Euclidean norm is not the physical $L^2(\Omega)$ norm. Let M be the finite-element mass operator, so that

$$\|v_N\|_{L^2(\Omega)}^2 := v^T M v.$$

The state errors of Exercise 1.3.2 are measured in this norm.

The quantity of interest is the average temperature on the heated edge,

$$Q(\mu) = \int_{\Gamma_N} u_N(x; \mu) ds = q^T u_N(\mu).$$

Because the same boundary functional defines the load vector, $q = F$.

Organize your code around an interface equivalent to

```

model = build_model(p=9, ell=5)

Av  = apply_A(mu, v, model)      # matrix-free A(mu) v
Aqv = apply_Aq(q_idx, v, model)  # matrix-free A_q v
Mv  = apply_M(v, model)          # matrix-free mass operator
u   = solve(mu, model)           # JAX CG
Q   = qoi(mu, model)             # Q(mu) = q^T u_N(mu)
mus = sample_mus(key, m, model)  # m parameters, uniform on D_p
U   = snapshots(mus, model)       # snapshot matrix, N x m
rom = make_rom(W, model)         # reduced operators for the basis W
Qr  = rom_qoi(mu, rom)           # reduced QoI Q_r(mu)

```

where

- `model` collects the mesh, the element connectivity, the element-to-block map, and the vectors F and q .
- `mu` is a parameter vector $\mu \in \mathcal{D}_p$ and `v` a vector of length N .
- `q_idx` is a block index $q \in \{1, \dots, p\}$, so that `apply_Aq` applies the single summand A_q .
- `key` is a JAX random key, `jax.random.key(seed)`¹, and `m` the number of parameters to draw. Draw the parameters separately from the solves. Exercise 1.2.3 reuses the *same* `mus` at three levels.
- `mus` is the resulting $m \times p$ array, and $U \in \mathbb{R}^{N \times m}$ has the columns $u_N(\mu^{(j)})$.
- W is a basis matrix $W_r \in \mathbb{R}^{N \times r}$ from the POD of Exercise 1.3.1. `rom` collects the reduced operators $A_{r,q}$, F_r , and q_r . Both appear only from Exercise 1.3 on.

The repeated functions `apply_A`, `solve`, and `qoi` must be JIT-compiled. A matrix-free Jacobi preconditioner is recommended.

Validate the implementation by running `solve` and `qoi` for every $p \in \{2, 4, 9, 16, 25\}$ using $\mu = c\mathbf{1}$. The exact solution and QoI are

$$u(x) = \frac{x_2}{c}, \quad Q = \frac{1}{c}.$$

Deliverable

Code only. Submit

- the element-to-block map and the matrix-free applications $A(\mu)v$, A_qv , and Mv .
- the JIT-compiled CG solve and QoI, and the snapshot routine.
- automated exact-solution tests for all five values of p .
- a README describing the data structures and where the affine decomposition enters.

Exercise 1.2: What is reducible? Sensitivity and snapshot evidence

1.2.1 Snapshot spectra for increasing p

For every $p \in \{2, 4, 9, 16, 25\}$, use $\ell = 5$ and draw at least $m = 1000$ independent parameters from $\text{Unif}([0.1, 10]^p)$. Generate the snapshot matrix

$$U = [u_N(\mu^{(1)}) \cdots u_N(\mu^{(m)})].$$

Compute the singular values and compare the normalized spectra σ_j/σ_1 in one semilogarithmic plot. Also report the smallest r satisfying

$$\frac{\sum_{j>r} \sigma_j^2}{\sum_j \sigma_j^2} \leq 10^{-4}.$$

¹Use `seed = 0` if not otherwise stated.

Deliverable

The following in the final report:

- one plot containing all five normalized spectra.
- a table with p , N , the wall time for the 1000 FOM solves, and the effective rank above.
- a 150–250 word discussion of how the state-space complexity changes with p .

1.2.2 Snapshot spectra under mesh refinement

Fix $p = 9$. Draw one set of $m = 1000$ parameters and reuse exactly the same parameters for

$$\ell \in \{4, 5, 6\}.$$

For each level, generate the snapshots and plot the normalized singular values σ_j/σ_1 . Compare the normalized spectra only. The absolute singular values carry the mesh-dependent scaling $\sigma_1 \sim h^{-1}$ and are not comparable across levels.

Deliverable

The following in the final report:

- one plot with the three normalized spectra.
- a 150–200 word discussion answering whether refinement substantially changes the practically relevant decay and effective rank.

Exercise 1.3: POD and Galerkin reduced models**1.3.1 Projection error and Galerkin ROM error**

Use the $\ell = 5$ snapshot sets from Exercise 1.2.2. For each $p \in \{2, 4, 9, 16, 25\}$, compute a Euclidean POD basis and form the Galerkin ROM

$$A_r(\mu)u_r(\mu) = F_r, \quad A_r(\mu) = \sum_{q=1}^p \mu_q A_{r,q},$$

where

$$A_{r,q} = W_r^T A_q W_r, \quad F_r = W_r^T F, \quad q_r = W_r^T q.$$

The full matrices A_q are still not required: compute $A_q W_r$ using your matrix-free operator.

Use an independent test set of at least 250 parameters and reduced dimensions from

$$r \in \{1, 2, 4, 6, 8, 12, 16, 24, 32\}$$

whenever available. For every p , plot the RMS relative state error of the Galerkin ROM versus r . For the default case $p = 9$, additionally compare

- the best Euclidean projection error $\|u - W_r W_r^T u\|_2 / \|u\|_2$,
- the actual Galerkin state error, and
- the QoI RMSE normalized by the standard deviation of the full-model QoI on the test set.

Discuss why the projection curve is a benchmark rather than the ROM error itself, and comment explicitly on the $p = 25$ case.

Deliverable

The following in the final report:

- one plot of the Galerkin state error versus r for all five parameter dimensions.
- one $p = 9$ plot comparing projection, ROM-state, and QoI errors.
- a 150–250 word discussion.

1.3.2 Runtime, speedup, and break-even

Use $\ell = 5$, and carry both the default $p = 9$ and the stress case $p = 25$ through the whole subsection. Because the FOM is matrix-free, there is no online global matrix-assembly stage. Measure separately:

- one matrix-free FOM application $A(\mu)v$.
- the full FOM solve plus QoI.
- for the ROM: affine reduced-operator combination, dense reduced solve, and reduced QoI evaluation.

Warm up JIT before timing and synchronize each measured result. Repeat timings and report a robust statistic such as the median.

For each p , plot online runtime versus r and combine the timing data with the test errors from Exercise 1.3.1 to make a runtime-versus-error plot. Then include the offline cost of snapshot generation, POD, and reduced-operator construction. If C_{off} is the offline cost and t_H, t_r are the FOM and ROM online times, estimate the break-even number of online queries

$$N_{\text{BE}} = \frac{C_{\text{off}}}{t_H - t_r}$$

when $t_H > t_r$. Report N_{BE} for both p , in each case at the smallest tested r that reaches a normalized QoI RMSE below 10^{-3} if possible. Explain whether reduction is still attractive when the parameter dimension is 25.

Deliverable

The following in the final report:

- a runtime-component plot versus r for both p .
- a runtime-versus-error plot.
- a table comparing $p = 9$ and $p = 25$ with the selected r , the online speedup, the offline cost, and the break-even query count.
- a 200–300 word discussion answering when the ROM pays off.

Exercise 1.4: Uncertainty quantification and multi-fidelity estimation

UQ setting

In this section the conductivity components are independent random variables,

$$\mu_q \sim \text{Unif}[0.1, 10], \quad q = 1, \dots, 9,$$

with $p = 9$ and $\ell = 5$. The target is the scalar expectation

$$\mathbb{E}[Q] = \int_{[0.1, 10]^9} Q(\mu) \rho(\mu) d\mu.$$

Here $Q = Q(\mu)$ is the quantity of interest of Exercise 1.1, made a random variable by $\mu \sim \rho$, with $\text{Var}(Q) := \mathbb{E}[(Q - \mathbb{E}[Q])^2]$. From Exercise 1.4.2 on, the subscript H marks the high-fidelity model $Q_H := Q$ and the subscript L the low-fidelity model $Q_L := Q_r$ of Exercise 1.3. A Monte Carlo estimator replaces this integral by an average over M independent parameters $\mu^{(1)}, \dots, \mu^{(M)}$ drawn from ρ ,

$$\mathbb{E}[Q] \approx \frac{1}{M} \sum_{i=1}^M Q(\mu^{(i)}).$$

The sampling error decays only as $M^{-1/2}$, independent of the parameter dimension, which makes cheap reduced models potentially valuable when many model evaluations are required.

1.4.1 Full-model Monte Carlo

Construct a reference estimate $Q_{\text{ref}} := \hat{Q}_{H, M_{\text{ref}}}$ from $M_{\text{ref}} \geq 10^4$ full-model samples. Report its standard error and the estimated standard deviation of the estimator given by

$$\hat{\text{se}} := \frac{s}{\sqrt{M_{\text{ref}}}}, \quad s^2 := \frac{1}{M_{\text{ref}} - 1} \sum_{i=1}^{M_{\text{ref}}} (Q(\mu^{(i)}) - Q_{\text{ref}})^2.$$

Every RMSE below is measured against Q_{ref} , not against the unknown $\mathbb{E}[Q]$. Keep $\hat{\text{se}}$ small compared with the smallest RMSE you plot. Then, for

$$M \in \{10, 30, 100, 300, 1000, 3000\},$$

repeat the FOM Monte Carlo experiment $R = 20$ times and estimate the RMSE of

$$\hat{Q}_{H, M} = \frac{1}{M} \sum_{i=1}^M Q(\mu^{(i)}).$$

Compare the empirical error with the prediction $\sqrt{\text{Var}(Q)/M}$. Use s^2 from the reference run for $\text{Var}(Q)$.

Deliverable

The following in the final report:

- one log-log plot of the empirical FOM Monte Carlo RMSE versus M , including the $M^{-1/2}$ prediction.
- a short discussion of the observed rate.
- the values of M_{ref} , Q_{ref} , and $\hat{\text{se}}$.

1.4.2 Monte Carlo with the ROM

Use the POD basis of Exercise 1.3.1 at $p = 9$, $\ell = 5$ and deliberately choose the modest dimension $r = 8$ so that the ROM approximation error remains visible. Define

$$\hat{Q}_{r, M} = \frac{1}{M} \sum_{i=1}^M Q_r(\mu^{(i)}).$$

Let

$$b_r = \mathbb{E}[Q_r - Q].$$

Derive the mean-squared error of $\hat{Q}_{r, M}$ with respect to the true $\mathbb{E}[Q]$. Estimate b_r from a large paired FOM-ROM sample and compare the predicted MSE with the empirical RMSE for the same sample sizes as in Exercise 1.4.1.

Deliverable

The following in the final report:

- the MSE derivation.
- one plot comparing FOM-only and ROM-only Monte Carlo error versus M , including the ROM bias floor and the theoretical curves.
- a 150–200 word discussion explaining why increasing M eventually stops helping the ROM-only estimator.

1.4.3 Control variates

A control variate uses a cheap model that is strongly correlated with the expensive model to reduce sampling variance. Here the ROM is not used as a replacement for the FOM. It is used only to correct the estimator statistically.

Use m_H paired FOM-ROM samples and a total of $m_L \geq m_H$ ROM samples. Consider

$$\hat{Q}_{cv} = \bar{Q}_H^{m_H} + \alpha (\bar{Q}_L^{m_L} - \bar{Q}_L^{m_H}),$$

where the first m_H low-fidelity samples use the same parameters as the high-fidelity samples and the remaining low-fidelity samples are independent.

Show that the estimator is unbiased. Let $\sigma_H^2 = \text{Var}(Q)$, $\sigma_L^2 = \text{Var}(Q_r)$, and $\rho = \text{Corr}(Q, Q_r)$. Derive its variance, the optimal control-variate coefficient, and the optimal sample ratio for per-sample costs c_H, c_L .

You may use the following reference formulas after you have derived them:

$$\alpha^* = \frac{\text{Cov}(Q, Q_r)}{\text{Var}(Q_r)},$$

$$\text{Var}(\hat{Q}_{cv}; \alpha^*) = \sigma_H^2 \left(\frac{1 - \rho^2}{m_H} + \frac{\rho^2}{m_L} \right),$$

$$\frac{m_L}{m_H} = \sqrt{\frac{\rho^2 c_H}{(1 - \rho^2) c_L}},$$

with the practical constraint $m_L \geq m_H$.

Deliverable

The following in the final report:

- the estimator and the proof of unbiasedness.
- the variance derivation and the optimal coefficient α^* .
- the optimal sample-allocation ratio.

This is a mathematical deliverable. No new plot is required in this subsection.

1.4.4 Multi-fidelity experiment

Estimate $\sigma_H, \sigma_L, \rho, c_H$, and c_L from pilot data. For a sequence of total runtime budgets, compare three estimators under approximately equal cost:

- FOM-only Monte Carlo.
- ROM-only Monte Carlo.
- the FOM-ROM control-variate estimator with α^* and an approximately optimal (m_H, m_L) allocation.

Use $R = 20$ independent replications to estimate RMSE. Plot RMSE against measured total runtime. Mark the ROM-only bias magnitude on the plot.

Deliverable

The following in the final report:

- one runtime-versus-RMSE plot containing all three estimators.
- a small table with the pilot estimates of $\rho, c_H/c_L, \alpha^*$, and the chosen allocations.
- a 200–300 word discussion.

Your discussion must explicitly explain why the control-variate estimator can achieve an error below the standalone ROM error while the ROM-only Monte Carlo estimator cannot.